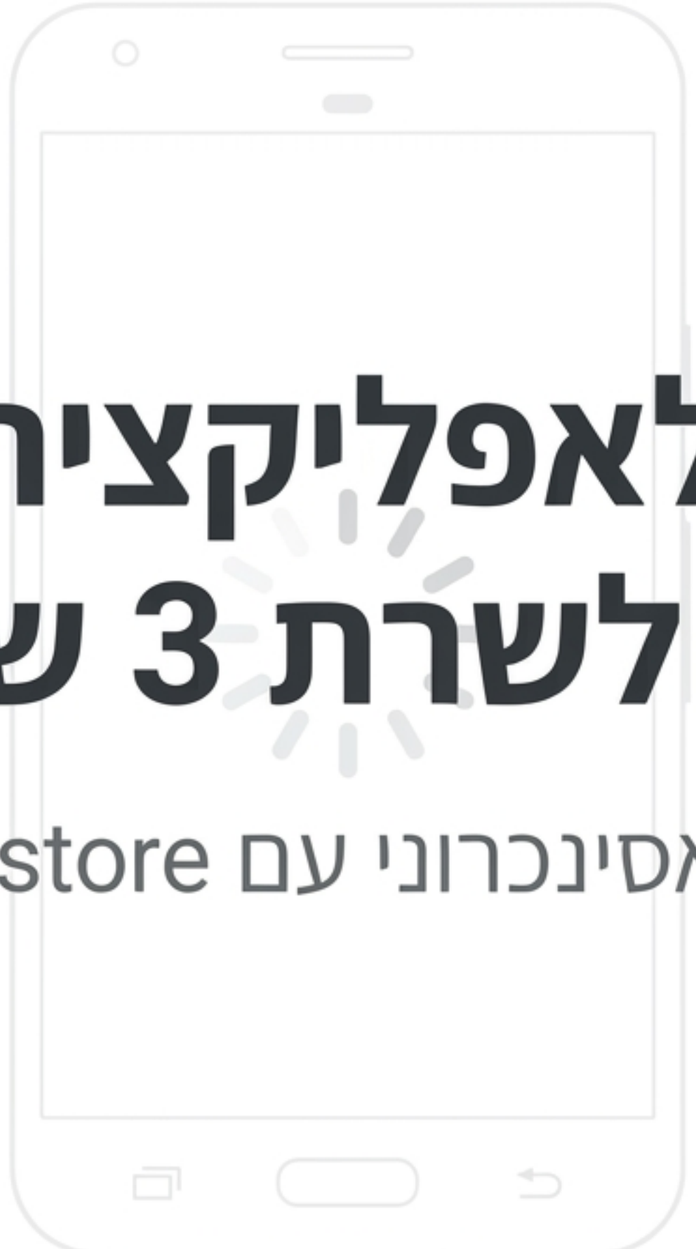


מה יקרה לאפליקציה שלנו אם נחכה לשרת 3 שניות?

מבוא לתכנות אסינכרוני עם Firebase Firestore

Gemini Notebook

ה-main thread:
עובד אחד, שתי משימות



- **כלל 1:** אסור לחסום אותו לעולם (למשל, המתנה לשרת).
- **כלל 2:** אסור לעדכן UI מחוץ לו.
- **הסכנה - ANR:** קריסת Application Not Responding. מתרחשת כשה-thread נחסם לכ-5 שניות. המסך קופא לחלוטין.

Gemini Notebook

סינכרוני

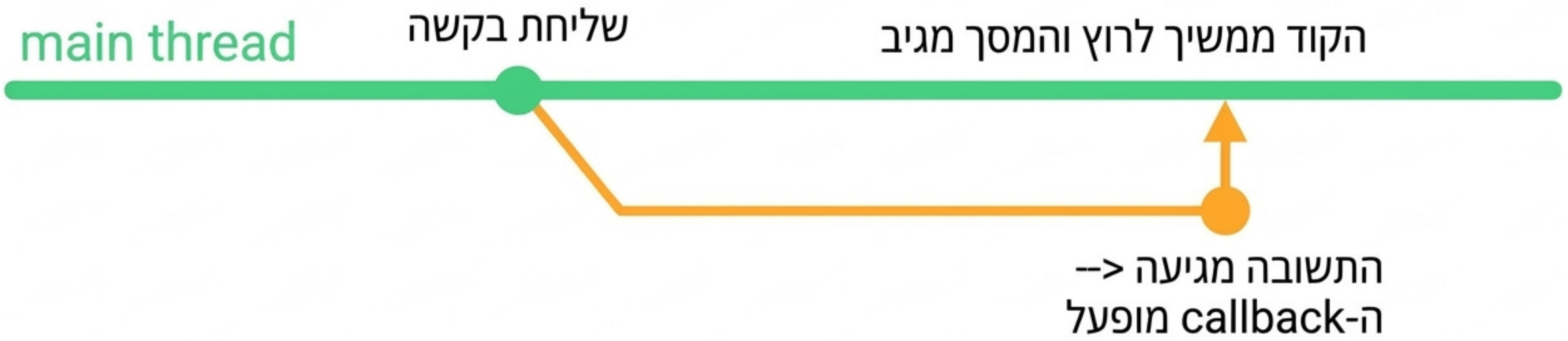


אסינכרוני



הזמזום הוא ה-Listener. במקום להמתין ולחסום את האפליקציה, אנחנו לוקחים זחזגום, ממשיכים הלאה, הלאה, והזמזום יקרא לנו כשהמידע מוכן!

Gemini Notebook



האפליקציה לא קופאת
בזמן ההמתנה לשרת.

Gemini Notebook

```java
private void loadScoreOnce() {
    Log.d("FS", "1: sending request");
    db.collection("players").document("p1").get()
        .addOnSuccessListener(new OnSuccessListener<DocumentSnapshot>() {
            @Override
            public void onSuccess(DocumentSnapshot doc) {
                Log.d("FS", "3: data arrived");
                tvScore.setText(String.valueOf(doc.getLong("score")));
            }
        });

    Log.d("FS", "2: method continues");
}
```

קריאה חד-פעמית: בקשה לקבלת הנתון פעם אחת.

הזמזם שלנו (Listener): הקוד שיופעל רק כאשר התשובה תגיע.

קסם ה-main thread: ה- callback של Firestore מופעלים אוטומטית על ה-main thread, לכן בטוח לעדכן כאן את ה-UI!

Gemini Notebook

שאלת חשיבה: מה יודפס קודם ב-Logcat?

```java
private void loadScoreOnce() {
    Log.d("FS", "1: sending request");
    db.collection("players").document("p1").get()
        .addOnSuccessListener(new OnSuccessListener<DocumentSnapshot>() {
            @Override
            public void onSuccess(DocumentSnapshot doc) {
                Log.d("FS", "3: data arrived");
                tvScore.setText(String.valueOf(doc.getLong("score")));
            }
        });
    Log.d("FS", "2: method continues");
}
```

Logcat Output

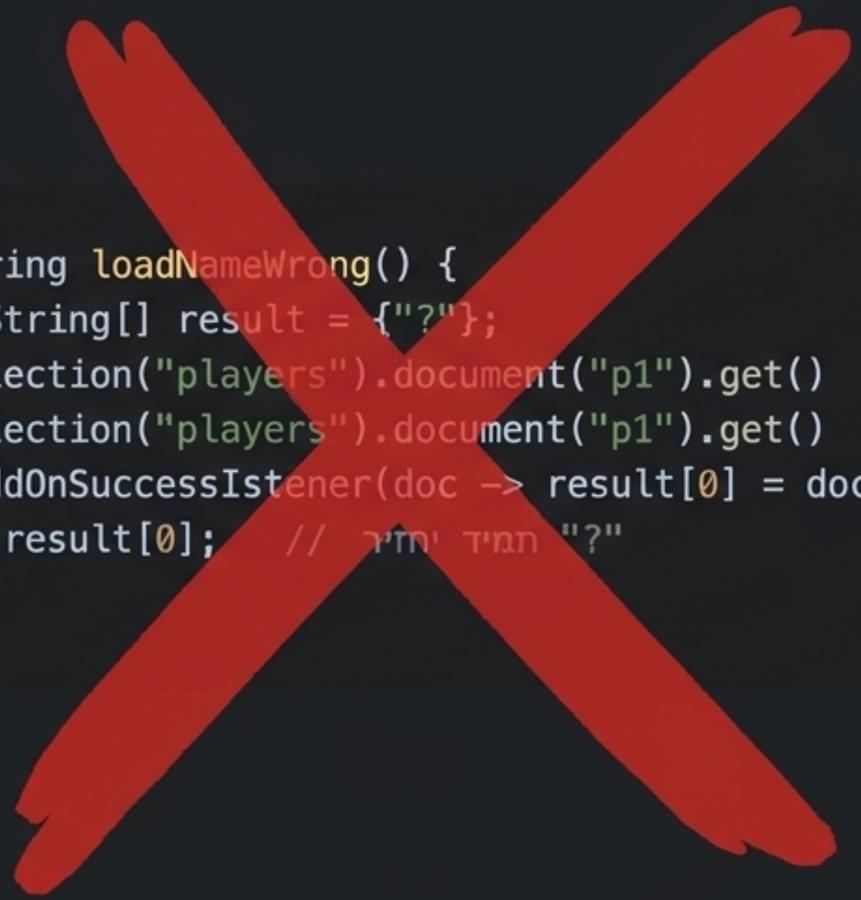
```
1: sending request
2: method continues
3: data arrived
```

למרות ששורה 3 כתובה בקוד לפני שורה 2, היא תופעל אחרונה. קוד ה-callback ממתין בעוד שהמתודה הראשית ממשיכה לרוץ מיד.

Gemini Notebook

מלכודת ה-return

```
private String loadNameWrong() {
    final String[] result = {"?"};
    db.collection("players").document("p1").get()
    db.collection("players").document("p1").get()
        .addOnSuccessIstener(doc -> result[0] = doc.getString("name"));
    return result[0];    // תמיד יחזיר "?"
}
}
```



1. משתנה result מאותחל ל-"?".
2. הבקשה נשלחת (אך התשובה טרם הגיעה).
3. הפקודה return מתבצעת לפני שה-callback הופעל!

הפתרון: אי אפשר לעשות return למידע אסינכרוני. חובה לעדכן את ה-UI בתוך ה-callback עצמו.

Gemini Notebook

```java
@Override
protected void onStart() {
    super.onStart();
    registration = db.collection("players").document("p1")
            .addSnapshotListener(new EventListener<DocumentSnapshot>() {
            @Override
            public void onEvent(DocumentSnapshot snap, FirebaseFirestoreExcep
                if (snap != null && snap.exists()) {
                    tvScore.setText(String.valueOf(snap.getLong("score")));
                }
            }
        });
}
}
```



עדכון בזמן אמת: addSnapshotListener

- במקום לקרוא פעם אחת, אנחנו פותחים צינור רציף.
- ה-callback יופעל מיד עם ההפעלה, ושוב בכל פעם שהמידע במסד הנתונים משתנה.
- המידע מגיע כ-snapshot (תמונת מצב) של המסמך באותו רגע.

Gemini Notebook

```java
    @Override
    protected void onStop() {
        super.onStop();
        if (registration != null) registration.remove();
    }
}
```



היגיינת מחזור חיים (Lifecycle)

- מאזין רציף שלא נסגר ימשיך לרוץ ברקע גם כשהאפליקציה ממוזערת.
- התוצאה: דליפת זיכרון (Memory Leak), בזבוז סוללה, וחיוב מיותר על קריאות רשת.
  רשת.
- הכלל: פותחים ב-onStart, ומנתקים (remove) (remove) ב-onStop.

Gemini Notebook

	addSnapshotListener (האזנה רציפה)	get() (קריאה חד-פעמית)
מתי להשתמש?	אפליקציית צ'אט, תוצאת משחק חי.	טעינת פרופיל משתמש בעת התחברות.
אנלוגיה	להשאיר את הדלת פתוחה ולקבל התראה על כל מכתב.	ללכת לבדוק את תיבת הדואר פעם אחת.
משאבים	דורש ניהול משאבים (חובה לנתק כדי למנוע בזבוז).	חסכוני (בקשה אחת = תשובה אחת).

Gemini Notebook

The Golden Rules

1

לעולם אל תחסמו את ה-main thread (השתמשו ב- Listener).

2

המידע קיים רק בתוך ה-callback (אל תנסו לעשות לו return).

3

תמיד נתקו מאזינים רציפים (השתמשו ב-remove).

אתגר מחשבה

שאלה לבית: נניח שאתם רוצים להציג אנימציית טעינה (ProgressBar) למשתמש. באילו שורות בדיוק בקוד של loadScoreOnce תציגו אותה (VISIBLE), ובאילו שורות תעלימו אותה (GONE)?

Gemini Notebook